Тема 5: Компютърни архитектури. Формати на данните. Вътрешна структура на ЦПУ.

ТЕОРИЯ

1. Обща структура на компютъра — принцип на запомнената програма

Принцип на запомнената програма (von Neumann): Програмата и данните се съхраняват в **една и съща** памет. Инструкциите са данни — могат да бъдат заредени, сменени, обработени.

Следствие: Една и съща машина изпълнява различни програми без смяна на хардуера.

Основни компоненти:

- **ЦПУ** (централен процесор) — извлича, декодира, изпълнява инструкции
- **RAM** (оперативна памет) — съхранява програмата и данните
- **Входно-изходни устройства** — клавиатура, дисплей, диск, мрежа
- **Системни шини** — адресна шина, шина за данни, управляваща шина

---

2. Концептуален цикъл на изпълнение на инструкция (Fetch-Decode-Execute)

Стъпка	Действие
Fetch	PC → адрес в памет; прочита се инструкцията в IR; PC се увеличава
Decode	Блокът за управление декодира IR — определя OpCode, операнди, адресен режим
Fetch operands	Извличат се операндите (от регистри, памет, или са непосредствени)
Execute	ALU или друг блок извършва операцията
Write back	Резултатът се записва в регистър или памет
Update PC	При обикновена инструкция PC вече е обновен; при преход — PC = нов адрес

> **PC** = Program Counter (програмен брояч); **IR** = Instruction Register

---

3. Формати на данните

3.1 Цели двоични числа

Беззнакови (unsigned): n бита → диапазон $[0, 2^n-1]$

- Стойност = $\Sigma\, b_i \cdot 2^i$
- 8 бита: 0…255; 16 бита: 0…65535; 32 бита: 0…4 294 967 295

Знакови — допълнителен код до 2 (Two's complement):

- Диапазон: $[-2^{n-1}, 2^{n-1}-1]$
- 8 бита: −128…127
- Положителни: нормален двоичен запис
- Отрицателни: инвертирай всички битове + 1
- **Предимства**: едно представяне на 0; събиране/изваждане с един хардуер
- **Знаков бит**: MSB = 1 → отрицателно

Пренос vs Препълване:

- **Пренос (Carry)** — при беззнакова аритметика резултатът надхвърля n бита
- **Препълване (Overflow)** — при знакова аритметика резултатът е невалиден (напр. +pos + +pos = negative)

3.2 Двоично-десетични числа (BCD — Binary-Coded Decimal)

- Всяка десетична цифра се кодира с **4 бита** ($0000_2 = 0 \ldots 1001_2 = 9$)
- Пример: 59_{10} = **0101 1001** BCD
- Използвани само 10 от 16 комбинации → неефективно
- **Предимство**: точно десетично представяне (без грешки при финансови изчисления)

3.3 Числа с плаваща запетая (IEEE 754)

Обща формула: **$(-1)^s \times 1.M \times 2^{(E-bias)}$**

Формат	Знак	Експонента	Мантиса	Bias
Single (32 бита)	1 бит	8 бита	23 бита	127
Double (64 бита)	1 бит	11 бита	52 бита	1023

- **Нормализация**: водещата 1 е имплицитна (hidden bit)
- **Bias**: позволява беззнаково сравнение на експоненти (exp = E − bias)
- **Специални стойности**: 0 (E=0, M=0), ±∞ (E=всички 1, M=0), NaN (E=всички 1, M≠0)
- **Проблем**: не всяко реално число е представимо → **грешки от закръгляне**

Пример — Single precision:

- 0 10000000 10000000000000000000000 = (+1) × 1.1_2 × 2^(128−127) = 1.5 × 2^1 = **3.0**

3.4 Символни данни и кодови таблици

Стандарт	**Бита**	**Обхват**
ASCII	7	128 символа: латиница, цифри, пунктуация, управляващи
Extended ASCII	8	256 символа
Unicode	—	> 1 100 000 кодови точки (всички писменост)
UTF-8	1–4 байта	ASCII-съвместимо, Variable Length Encoding
UTF-16	2–4 байта	2 байта за BMP символите

UTF-8 принцип: ASCII символите (0–127) = 1 байт; кирилица = 2 байта; CJK = 3 байта.

---

4. Вътрешна структура на ЦПУ

4.1 Регистри

Вид регистър	**Роля**
GPR (General Purpose)	Временни операнди и резултати (eax, ebx, rcx…)
PC / IP	Адрес на следващата инструкция
IR	Текущо декодираната инструкция
SP (Stack Pointer)	Върха на стека
BP / FP (Base/Frame Pointer)	База на стековата рамка
FLAGS / PSW	Регистър на състоянието (флагове)
Сегментни (x86)	CS, DS, SS, ES — базови адреси на сегменти

Ширина: 8/16/32/64 бита (x86: AL/AX/EAX/RAX — части на един регистър).

4.2 Аритметико-логическо устройство (ALU)

Изпълнява:

- **Аритметика**: ADD, SUB, MUL, DIV, INC, DEC, NEG
- **Логика**: AND, OR, XOR, NOT
- **Измествания**: SHL, SHR, SAR (arithmetic right), ROL, ROR
- **Сравнение**: CMP (изваждане без запис, обновява флаговете)

ALU получава два входа (от регистри/памет), извършва операцията и записва резултат +

обновява FLAGS.

4.3 Регистър на състоянието (FLAGS / EFLAGS / RFLAGS) и флагове

Флаг	**Символ**	**Значение**
Zero Flag	ZF	Резултатът е 0
Sign Flag	SF	MSB на резултата = 1 (отрицателен)
Carry Flag	CF	Пренос при беззнакова аритметика
Overflow Flag	OF	Препълване при знакова аритметика
Parity Flag	PF	Четен брой 1-битове в резултата
Direction Flag	DF	Посока на низови инструкции (0=нагоре, 1=надолу)
Interrupt Flag	IF	Разрешаване на прекъсвания

4.4 Блок за управление (Control Unit)

- **Декодира** инструкциите (OpCode → управляващи сигнали)
- **Координира** ALU, регистри, памет, I/O
- **Два вида реализация**:
  - **Hardwired (твърда логика)**: комбинационни схеми → бързо, трудно за промяна
  - **Microprogrammed (микропрограмно)**: всяка инструкция = микропрограма от микроинструкции → гъвкаво, малко по-бавно

---

5. Конвейерна обработка (Pipeline)

Идея: Разделяне на изпълнението на инструкция на **k етапа**; паралелно изпълнение на k инструкции (всяка в различен етап).

Типични етапи (5-степенен конвейер):

```
IF → ID → EX → MEM → WB
(Fetch  Decode  Execute  Memory  WriteBack)
```

Без конвейер: N инструкции × k такта = N·k такта
С конвейер: k + (N−1) такта ≈ N такта при голямо N

Конфликти (Hazards):

Тип	**Причина**	**Решение**
Structural	Два етапа искат един ресурс	Дублиране на ресурс, stall

Тип	Причина	Решение
Data	Инструкция чака резултат от предишна	Forwarding/bypassing, stall (bubble)
Control	Условен преход — следваща инстр. неизвестна	Branch prediction, pipeline flush

---

6. Инструкции на ЦПУ

6.1 Формат на инструкция

```
[Prefix] | Opcode | ModRM | SIB | Displacement | Immediate
```

Prefix (незадължителен): modifier за размер на операнда (66h), размер на адреса (67h), REP/REPE/REPNE за низови инструкции, LOCK.

Opcode: 1–3 байта; определя операцията.

ModRM байт: задава режима на адресация и операндите.

SIB байт (Scale-Index-Base): за сложни адресни режими.

6.2 Местоположение на операндите

Тип	Описание
Непосредствен (Immediate)	Стойността е в самата инструкция
Регистров	Операндът е в регистър
В паметта	Операндът е на адрес в RAM
I/O порт	Четене/запис от хардуерен порт

6.3 Модели на адресация

Режим	Ефективен адрес	Пример (x86)
Непосредствен	стойността е операндът	`MOV EAX, 5`
Регистров	операндът е в регистъра	`MOV EAX, EBX`
Директен	EA = адрес в инструкцията	`MOV EAX, [1000h]`
Индиректен	EA = съдържание на регистър	`MOV EAX, [EBX]`
Базов	EA = Base + offset	`MOV EAX, [EBX+8]`
Индексен	EA = Base + Index×Scale	`MOV EAX, [EBX+ECX*4]`
Базово-индексен	EA = Base + Index×Scale + disp	`MOV EAX, [EBX+ECX*4+8]`
Стеков	EA = SP (PUSH/POP)	`PUSH EAX`

6.4 Аритметико-логически инструкции

```
ADD dst, src     ; dst = dst + src, обновява CF,OF,ZF,SF
SUB dst, src     ; dst = dst − src
MUL src          ; беззнаково умножение (EAX × src)
IMUL src         ; знаково умножение
DIV src          ; беззнаково деление
IDIV src         ; знаково деление
INC dst          ; dst++
DEC dst          ; dst--
NEG dst          ; dst = −dst
AND dst, src     ; побитово AND
OR  dst, src     ; побитово OR
XOR dst, src     ; побитово XOR
NOT dst          ; побитово NOT
SHL dst, n       ; логическо ляво изместване (×2ⁿ)
SHR dst, n       ; логическо дясно изместване (÷2ⁿ)
SAR dst, n       ; аритметично дясно (запазва знака)
CMP dst, src     ; dst−src без запис, само флагове
TEST dst, src    ; dst AND src без запис, само флагове
```

6.5 Низови инструкции

Работят с SI/DI (или RSI/RDI) като указатели, CX/ECX/RCX като брояч, DF флага за посока.

```
MOVSB/MOVSW/MOVSD   ; копиране: [DS:SI] → [ES:DI], ±1/2/4
CMPSB/CMPSW/CMPSD   ; сравнение: [DS:SI] − [ES:DI]
SCASB/SCASW/SCASD   ; търсене: AL/AX/EAX − [ES:DI]
STOSB/STOSW/STOSD   ; запис: AL/AX/EAX → [ES:DI]
LODSB/LODSW/LODSD   ; зареждане: [DS:SI] → AL/AX/EAX
```

Префикс REP: повтаря инструкцията ECX пъти (ECX-- след всяка).
REPE/REPZ: повтаря докато ZF=1.
REPNE/REPNZ: повтаря докато ZF=0.

6.6 Безусловни и условни преходи

Безусловни:

```
JMP адрес        ; PC = адрес (короткий/дальний)
CALL адрес       ; PUSH PC; JMP адрес (извикване на подпрограма)
RET              ; POP PC (връщане от подпрограма)
```

Условни (след CMP или друга инструкция, обновила флаговете):

Инструкция	**Условие**	**Флагове**
JZ / JE	равно / нула	ZF=1
JNZ / JNE	не равно	ZF=0
JL / JNGE	по-малко (знаково)	SF≠OF
JG / JNLE	по-голямо (знаково)	ZF=0 и SF=OF
JB / JNAE	по-малко (беззнаково)	CF=1
JA / JNBE	по-голямо (беззнаково)	CF=0 и ZF=0
JO	overflow	OF=1
JS	знак отрицателен	SF=1

6.7 Управление на програмата

Подпрограми (стек):

```
CALL proc    →  [SP-4] = PC_next;  SP -= 4;  PC = proc
RET          →  PC = [SP];  SP += 4
```

Стекова рамка (calling convention):

```
PUSH EBP          ; запази базовия указател
MOV EBP, ESP      ; нова стекова рамка
SUB ESP, N        ; резервирай N байта за локални промен.
...
MOV ESP, EBP      ; освободи локалните
POP EBP           ; възстанови базовия указател
RET
```

Прекъсвания (Interrupts):

- Хардуерни (IRQ) — от I/O устройства
- Софтуерни (INT n) — системни извиквания
- Изключения (exceptions) — деление на 0, page fault и т.н.

Бърза справка

Понятие	**Ключ**
Two's complement −x	Инвертирай + 1
BCD	4 бита / десетична цифра
IEEE 754 single	1+8+23 бита, bias=127
UTF-8	Variable-length, ASCII-съвместимо
Pipeline конфликти	Structural / Data / Control
ZF=1 след CMP	операндите са равни
CF=1 след ADD	беззнаков пренос
OF=1 след ADD	знаково препълване
CALL	PUSH return addr + JMP
REP MOVSB	копира ECX байта DS:SI → ES:DI